

Week 5 — Classic IP Models, Networks, and the LAP

OL7014 Integer Programming • Instructor: Prof. H. Park • Week of Sep 21, 2026

1. Matching — a model born canonical

demo `ip-games / assignment` — play FIRST: three pilots, three aircraft, one each. <https://hankpark0706.github.io/ip-games/assignment-matching.html>

3 pilots, 3 aircraft, one each — or officers to billets, units to sectors, interpreters to teams. c_{ij} = value of the pairing (i, j) . this is the **assignment problem** (할당 문제) — a perfect *matching* in a bipartite graph — and the decision is a *table*:

$$X \in \{0, 1\}^{3 \times 3}, \quad x_{ij} = 1 \text{ iff pilot } i \text{ flies aircraft } j$$

rows sum to 1 (each pilot once), columns sum to 1 (each aircraft once):

$$\begin{aligned} \max_X \quad & \langle C, X \rangle \\ \text{s.t.} \quad & X\mathbf{1} = \mathbf{1}, \quad X^\top \mathbf{1} = \mathbf{1}, \quad X \geq 0 \end{aligned}$$

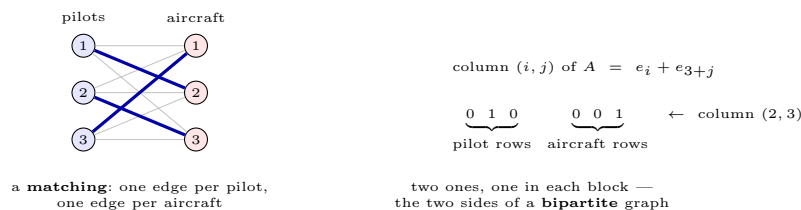
read the symbols: $\langle C, X \rangle = \sum_{i,j} c_{ij}x_{ij}$, $\mathbf{1}$ = all-ones vector

matrix form by **flattening**: $x = \text{vec}(X) \in \mathbb{R}^9$, $c = \text{vec}(C) \Rightarrow$ a 6×9 matrix of 0/1, $b = \mathbf{1}$

- each column of A : exactly **two ones** — one pilot row, one aircraft row (an *incidence matrix*)
- binaries already gone: row sums = 1 with $x \geq 0$ cap every variable at 1
- rank $A = 5$: any one row is implied by the rest

now run week 2's experiment again: delete integrality, solve the LP, look at the answer

it comes back a matching — the LP always has an *integral* optimum, and a vertex solver always returns one. no cost table breaks it



KEY $2n$ rows, n^2 variables, a harder-looking model than week 1's two-row toy — and an *easier* answer. something structural is going on.

takeaway matching's LP returns whole numbers with no integrality declared. the reason has a name — *total unimodularity* — and it is the next section.

2. Transportation — matching with quantities

assignment said: each source supplies exactly one unit, each sink needs exactly one. relax that — source i holds supply s_i , sink j needs demand d_j — and matching becomes **flow**

quantities are continuous, so by the course convention the variable is a y : y_{ij} = amount shipped $i \rightarrow j$

$$\begin{aligned} \min_y \quad & \sum_{i=1}^n \sum_{j=1}^k c_{ij} y_{ij} \\ \text{s.t.} \quad & \sum_{j=1}^k y_{ij} \leq s_i \quad (i \in [n]) \quad \text{ship no more than you hold (공급 한도)} \\ & \sum_{i=1}^n y_{ij} \geq d_j \quad (j \in [k]) \quad \text{every sink gets what it needs (수요 충족)} \\ & y_{ij} \geq 0 \end{aligned}$$

balanced case — nothing left over: $\sum_i s_i = \sum_j d_j$

with shipping costs $c \geq 0$ nobody ever ships spare, so some optimum has every $\geq d_j$ row tight — which is why they may be written as =

assignment is the special case hiding inside: $k = n$, every $s_i = d_j = 1$

the row sums then cap every y_{ij} at 1, and the next section puts the optimum on a *corner* — which is a permutation matrix. only the letter changes, $y \rightarrow x$

look at what this model does not contain: a single integer variable. it is a plain LP

and yet: integer $s_i, d_j \Rightarrow$ the optimal shipments come out integer. same phenomenon as matching, same reason

takeaway transportation = assignment with quantities instead of ones: supplies left, demands right, flows between. no binaries, and integral for free when the data is.

3. Total unimodularity — why the LP told the truth

week 1's toy: the LP said 22.5 at (2.5, 2.5), the truth was 19 at (1, 3) — the relaxation lied. matching: the same experiment returns a matching, exactly, every time

same solver, same kind of 0/1 question $\Rightarrow$ the difference is in the *matrix*

Def. A is **totally unimodular (TU)** (완전 유니모듈러 행렬) if *every* square submatrix of A has determinant 0, +1, or -1

square submatrix: pick any k rows and any k columns, throw the rest away, take the determinant. every one of them, every k

note what the definition does *not* mention: b , c , integrality — TU is a property of the coefficient matrix alone
 $k = 1$ **is free**: a 1×1 submatrix is a single entry, and its determinant is itself

$\Rightarrow$ every entry of a TU matrix is 0, +1, or -1

one glance now settles week 2's rucksack: its row was (5, 7, 8, 8, 8, 2) — there is an 8 in it, so *that* matrix is not TU

(it is the numbers, not the shape: a pure counting row $\sum_i x_i \leq p$ is all ones, and all ones is TU)

matching's matrix is all 0/1, so it at least survives the first test

Prop. (Hoffman–Kruskal) A TU and b integer $\Rightarrow$ every corner of $P = \{x : Ax \leq b, x \geq 0\}$ has integer coordinates

(same for $\{x : Ax = b, x \geq 0\}$). hence the LP $\max\{c^\top x : x \in P\}$, whenever it has an optimum at all, has an integer one:

the LP relaxation solves the IP — no branching, no rounding, no integrality constraint required

the mechanism, in one paragraph: in equality form $Ax = b, x \geq 0$, a corner is given by a choice of *basic columns* B — the rest are 0 — and x_B solves the square system $A_B x_B = b$. Cramer's rule:

$$(x_B)_j = \frac{\det(A_B \text{ with its } j\text{th column replaced by } b)}{\det A_B}$$

integer data puts an integer on top; $\det A_B = \pm 1 \Rightarrow$ the division cannot manufacture a fraction. (a determinant of 0 means those columns were not a basis — they never described a corner.)

KEY TU is the promise that *no corner of your polyhedron can be fractional, no matter which integer b you throw at it.*

4. Recognizing it — two dimensions, then the checklist

keep week 1's objective $\max 4x + 5y$ and $x, y \geq 0$ fixed; change *nothing but* A :

$$(P_0) \begin{cases} x + 3y \leq 10 \\ 3x + y \leq 10 \end{cases} \quad (P_1) \begin{cases} x + y \leq 5 \\ -x + y \leq 2 \end{cases} \quad (P_2) \begin{cases} x + y \leq 5 \\ y \leq 2 \end{cases}$$

(P_0) = week 1's toy. (P_1) : two teams fly at most 5 sorties between them, and team B at most two *more than* team A. (P_2) : replace that comparison with a flat cap — team B flies at most two

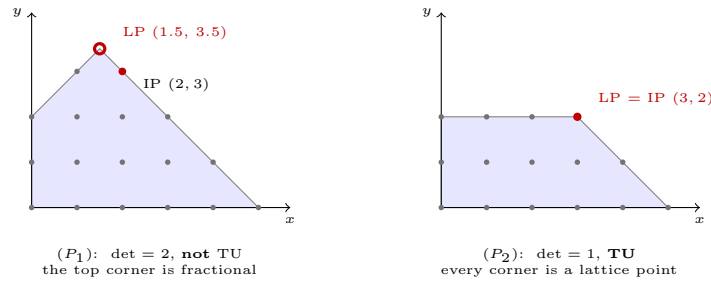
	A	largest det	TU?	LP optimum	IP optimum
(P_0)	$\begin{pmatrix} 1 & 3 \\ 3 & 1 \end{pmatrix}$	8	no — there is a 3	(2.5, 2.5), 22.5	(1, 3), 19
(P_1)	$\begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}$	2	no — det = 2	(1.5, 3.5), 23.5	(2, 3), 23
(P_2)	$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$	1	yes	(3, 2), 22	(3, 2), 22

(week 1 wrote these with the rows $-x \leq 0, -y \leq 0$ stacked underneath; appending them changes neither the largest |det| nor any TU verdict)

read the last two rows against each other — that is the whole lesson. (P_1) and (P_2) differ in **one entry**

$\det \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} = 1 - (-1) = 2$, and a determinant of 2 is a division by 2 waiting to happen — there is your 1.5

change that -1 to a 0: the determinant drops to 1, the corners snap onto integers, and the LP hands back (3, 2) with no integrality anywhere in the model



two traps.

1. a matrix of 0s and ± 1 s need *not* be TU — “all my coefficients are 0/1, so my LP will come out integral” is a wish, not a theorem. (P_1) is the counterexample

2. TU is a property of the matrix *as a whole*, not of its rows: neither row of (P_1) is guilty of anything, the 2 appears only when you put them together. you cannot audit a model for TU one constraint at a time

so how do you actually check? not by determinants — a 6×9 matrix already has thousands of square submatrices. one checklist covers every model in this course (*sufficient*, not necessary):

- (i) every entry is $0, \pm 1$
- (ii) every column has at most two nonzeros
- (iii) the rows split into M_1, M_2 so that **in every column with two nonzeros** the M_1 entries and the M_2 entries sum to the same thing

all three $\Rightarrow$ TU. and the whole course sits in two ways of passing (iii):

• **two +1s in a column** (an *undirected* incidence matrix): the two must land on opposite sides, so M_1, M_2 is a split of the nodes with every edge crossing it — *exactly bipartiteness*

assignment’s pilot rows and aircraft rows **are** M_1 and M_2 : *the incidence matrix of a bipartite graph is TU*

• **one +1 and one -1** (a *directed* incidence matrix): take $M_1 =$ all rows, $M_2 = \emptyset$; $(+1) + (-1) = 0$ and the empty sum is 0 — the networks below, and no condition on the graph at all

transportation has that same shape, sources on one side and sinks on the other, so it inherits the guarantee:

model	matrix	LP relaxation
week 1’s toy (P_0)	a row with a 3 in it	fractional corner $(2.5, 2.5)$
assignment	bipartite incidence, TU	integral: the LP <i>is</i> the IP
transportation	bipartite incidence, TU	integral whenever s, d are

same fact in the other vocabulary: the feasible set of assignment is the *doubly stochastic* matrices, and its corners are exactly the permutation matrices — the honest matchings

demo `demo1_toy_ip.py` — week 2’s demo on week 1’s toy, re-read with this week’s eyes: the `r1, r2` sliders move b , but there is no slider for A — and A is where TU lives. set `r1 = r2 = 4` and the LP optimum does land on $(1, 1)$: failing TU means *some* integer b gives fractions, not every one. TU is the promise about all of them at once.

takeaway first time in this course that the *shape* of A — not its size, not the solver — decided whether a problem was easy.

5. Minimum cost network flow (MCNF) — the family worth recognizing

demo `ip-games / min-cost-flow` — push flow through a network by hand, then let the solver do it. <https://hankpark0706.github.io/ip-games/min-cost-network-flow.html>

directed graph $G = (V, E)$: $V = \{1, \dots, m\}$ nodes (노드), $E \subseteq \{(i, j)\}$ arcs (아크)

at each node i : $b_i =$ net supply = outflow – inflow

$b_i > 0$	node i supplies	source (공급지)
$b_i < 0$	node i demands	destination (수요지)
$b_i = 0$	node i only passes flow through	transshipment , a hub (환적지)

on each arc (i, j) : $c_{ij} =$ unit shipping cost, $l_{ij} \leq y_{ij} \leq u_{ij}$ (bounds; 0 and ∞ unless the road says otherwise)

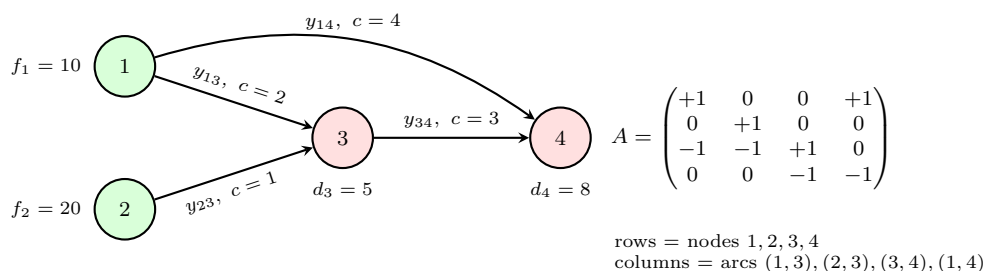
decision: $y_{ij} =$ flow on arc (i, j)

$$\begin{aligned}
 \min_y \quad & \sum_{(i,j) \in E} c_{ij} y_{ij} \\
 \text{s.t.} \quad & \underbrace{\sum_{j: (i,j) \in E} y_{ij} - \sum_{j: (j,i) \in E} y_{ji}}_{\text{one row per node — flow conservation (흐름 보존)}} = b_i \quad (\forall i \in V) \\
 & l_{ij} \leq y_{ij} \leq u_{ij}
 \end{aligned}$$

balance is forced: summing all rows gives 0 on the left $\Rightarrow \sum_{i \in V} b_i = 0$ is required

if the data does not balance, add a **dummy node** with zero-cost arcs to absorb the difference

the matrix is the node-arc incidence matrix: one row per node, one column per arc, +1 where the arc leaves, -1 where it arrives



run the checklist above on it: entries $0, \pm 1$ ✓, two nonzeros per column ✓, $M_1 = V$ and $M_2 = \emptyset$ ✓ — every directed graph passes, with no condition on the graph

(an *undirected* edge would put two +1s in its column, and then only a bipartite split passes — which is why the **matching LP** is integral only when the graph is bipartite)

the four rows above add to the zero row: $\det A = 0$ and $\text{rank } A = 3 = |V| - 1$ — one conservation row is always redundant

Prop. integer supplies $\{b_i\}$ and integer arc bounds $\{l_{ij}\}, \{u_{ij}\} \Rightarrow$ every extreme point of MCNF is integral: the LP solves the IP, **no branching, ever**

one model, four classics:

problem	is MCNF with
transportation	sources $b_i = s_i > 0$, destinations $b_j = -d_j$
assignment	transportation with every $b_i = +1$, $b_j = -1$ (equally many of each)
shortest path	$b_s = +1$, $b_t = -1$, $b_i = 0$ otherwise
max flow	every node transshipment ($b_i = 0$); add an artificial arc (t, s) and maximize y_{ts}

so assignment and transportation were the same model twice — and both were networks

the picture above is not yet an MCNF instance. nodes 3 and 4 are destinations ($b_3 = -5$, $b_4 = -8$), so nodes 1 and 2 must supply 13 between them — *which one, and how much, is not data*. that is the LAP, two sections down

KEY if you can recognize an IP as a network, it is solved: one row per node, one column per arc, ± 1 entries — TU by construction.

6. Facility location — paying to open

transportation assumed the depots were already there, holding s_i tons apiece. the planning question that actually costs money comes first: *which depots should exist at all?* (시설 입지 문제)

one new decision, one new cost: $x_i = \begin{cases} 1 & \text{depot } i \text{ opens} \\ 0 & \text{otherwise} \end{cases}$ $f_i = \text{fixed cost of opening depot } i$

$$\begin{aligned} \min_{x, y} \quad & \sum_i f_i x_i + \sum_i \sum_j c_{ij} y_{ij} && \text{fixed cost + shipping cost} \\ \text{s.t.} \quad & \sum_i y_{ij} = d_j && \text{(demand)} \\ & y_{ij} \leq M x_i && \text{(linking — week 2)} \\ & y_{ij} \geq 0, \quad x_i \in \{0, 1\} \end{aligned}$$

read the linking row: $x_i = 0 \Rightarrow y_{ij} = 0$ (a closed depot ships nothing); $x_i = 1$ releases it entirely — which is what *uncapacitated* means

how big is M? week 2's craft applies on the spot: this row governs one (i, j) pair, and no plan ever ships more than d_j to sink j , so $M = d_j$ is valid and smallest. ($M = \sum_j d_j$ is also valid — and on a two-depot instance costs 5 points of LP bound for nothing.)

1. fix x and transportation falls out. if somebody tells you which depots are open, $\sum_i f_i x_i$ is a constant and the linking rows become plain bounds; what is left is transportation. *all the hardness is in the binaries*

2. the big- M row costs you the guarantee. transportation's matrix was a bipartite incidence matrix; the row $y_{ij} \leq M x_i$ belongs to no incidence matrix at all. the MCNF proposition no longer applies

by trap 1 that is not yet a proof that the LP *will* lie — so the next section goes and checks

KEY every guarantee in this note is a property of A — so every guarantee is one modeling decision away from being lost.

7. The LAP — when b is a decision

MCNF takes the supplies b_i as data. the **location–allocation problem** (입지-배분 문제) does not: *we choose them*
 $r_i \geq 0$ = reserve pre-positioned at node i (holding cost h_i /ton), $q_i \geq 0$ = unmet demand (penalty u_i), $s_i \geq 0$ = surplus (penalty o_i)

$$b_i = \underbrace{r_i}_{\text{decision}} - d_i + q_i - s_i \implies \underbrace{\sum_j y_{ji} - \sum_j y_{ij}}_{\text{inflow} - \text{outflow}} + r_i + q_i - s_i = d_i \quad (\forall i \in V)$$

that is the whole difference between a network flow problem and the LAP: b is not given to us

$$\begin{aligned} \min_{x, r, y, q, s} \quad & \sum_i f_i x_i + \sum_i h_i r_i + \sum_{(i,j) \in E} c_{ij} y_{ij} + \sum_i u_i q_i + \sum_i o_i s_i \\ \text{s.t.} \quad & \sum_j y_{ji} - \sum_j y_{ij} + r_i + q_i - s_i = d_i & (\forall i) & \quad [\text{conservation} - \text{MCNF}] \\ & \sum_{i \in S} r_i \leq B & & \quad [\text{budget} - \text{one knapsack row}] \\ & r_i \leq B x_i & (\forall i \in S) & \quad [\text{linking} - \text{week 2}] \\ & \sum_{i \in S} x_i \leq p & & \quad [\text{at most } p \text{ depots} - \text{week 2}] \\ & y, r, s \geq 0, \quad 0 \leq q_i \leq d_i, \quad x_i \in \{0, 1\} \end{aligned}$$

five costs: open (f_i), hold (h_i), ship (c_{ij}), fail (u_i), waste (o_i)

$S \subseteq V$ = the **candidate sites**; elsewhere $x_i = r_i = 0$ — you do not stock reserve inside the disaster area
every constraint is old, and $M = B$ passes in the linking row because the budget row already caps every r_i at B — valid, though week 2 would ask whether it is the *smallest* valid M

the network above, priced — $S = \{1, 2\}$, $B = 15$, $p = 1$, $h_i = 1$, $u_i = 100$, $o_i = 1$:

$$\begin{aligned} \min \quad & 10x_1 + 20x_2 + r_1 + r_2 + 2y_{13} + y_{23} + 3y_{34} + 4y_{14} + 100(q_3 + q_4) + \sum_i s_i \\ \text{s.t.} \quad & -y_{13} - y_{14} + r_1 + q_1 - s_1 = 0 & [\text{node 1}] \\ & -y_{23} + r_2 + q_2 - s_2 = 0 & [\text{node 2}] \\ & y_{13} + y_{23} - y_{34} + q_3 - s_3 = 5 & [\text{node 3}] \\ & y_{34} + y_{14} + q_4 - s_4 = 8 & [\text{node 4}] \\ & r_1 + r_2 \leq 15, \quad r_1 \leq 15x_1, \quad r_2 \leq 15x_2, \quad x_1 + x_2 \leq 1 \\ & y, r, s \geq 0, \quad 0 \leq q_3 \leq 5, \quad 0 \leq q_4 \leq 8, \quad x_1, x_2 \in \{0, 1\} \end{aligned}$$

sixteen variables, two of them binary — and $q_1 = q_2 = 0$ is forced by $q_i \leq d_i$, so fourteen really move. every number came off the picture. at $p = 1$ there are two plans:

plan	open	hold	ship	total
$x_2 = 1$: $r_2 = 13$, $y_{23} = 13$, $y_{34} = 8$	20	13	$1 \cdot 13 + 3 \cdot 8 = 37$	70
$x_1 = 1$: $r_1 = 13$, $y_{13} = 5$, $y_{14} = 8$	10	13	$2 \cdot 5 + 4 \cdot 8 = 42$	65

site 2 ships cheaper and still loses: it would save 5 on the road and pay 10 more at the door

the gap is entirely city 3 (5 tons at 1/ton against 2/ton); on city 4 the two sites are exactly tied — 4/ton direct from site 1 against 1 + 3 through the hub from site 2

raise f_1 to 15 and the two tie at 70; past that, or with the bypass arc (1, 4) flooded (site 1 then pays $2 \cdot 13 + 3 \cdot 8 = 50$ to ship, total 73), site 2 wins

that trade — a cost paid once against a cost paid per ton — is the LAP

and the LP? it lies, exactly as the big- M row warned:

$$z_{\text{LP}} = 62 < z_{\text{IP}} = 65, \quad \text{at } x_1 = \frac{8}{15}, \quad x_2 = \frac{1}{3}, \quad r_1 = 8, \quad r_2 = 5$$

it buys eight-fifteenths of depot 1 and a third of depot 2, pays $10 \cdot \frac{8}{15} + 20 \cdot \frac{1}{3} = 12$ to open instead of 10, then serves city 3 from site 2 at 1/ton *and* city 4 from site 1 at 4/ton — a shipping bill of 37 that no single depot can achieve. $12 + 13 + 37 = 62$

half-open depots are not a plan, and the 12 is not a price anyone can pay. *that* is what losing TU costs

demo `lap_app.py` — this model on a map — and a head start on week 9, since the dashboard fits the plan to *several* sampled disasters: drag **training data** down to 1 and it collapses to today's single-scenario model. **max facilities** is p , and the prices are B (budget **B**), f_i (facility cost **c**), h_i (reserve cost **f**) — the app's letters do not line up with ours. *Solve* picks the plan, *Simulate disaster* routes it against one demand draw; leave *SAA vs mean* alone, that button is week 9.

takeaway **LAP = MCNF + two decisions MCNF cannot make: which nodes supply (x_i) and how much (r_i).**
fix those two and a network flow problem is all that is left — TU, easy. all the hardness is in (x, r).

8. The assumption we just made

look at where d_i sits in the model: on the right-hand side, *known*. and look at when we choose x_i and r_i : in the same model, at the same moment, as y_{ij} and q_i

so the model assumes you know each city's demand *when you decide where to build*. nobody knows where the earthquake will be

the real sequence has a gap:

stage 1: now	$\longrightarrow$ the disaster $\longrightarrow$	stage 2: then
open depots, place stock	demand d reveals itself	route what you have
x_i, r_i		y_{ij}, q_i, s_i
committed, binary, expensive		reactive, a flow problem, easy

the split is the one the model above already found *in the variables* — the model does not need rebuilding, it needs re-reading

what “optimal” means across that gap is the open question: plan for the worst case and you build for an earthquake that never comes; plan for the average and note that the average disaster never happens

no formulation today — that is week 9, and it needs the weeks in between first

KEY *ML predicts; optimization decides* — a forecast of d is an input, not an answer.

next lecture is $r_i \leq Bx_i$ the *best* way to write that row? other formulations describe the same plans and return the same answer — and one solves in seconds where another runs all night. **Homework 2 is out** (due 10/6).